Friedrich-Alexander-Universität
Technische Fakultät

Pattern Recognition Lab

FAU

Introduction to Software Engineering

Software Evolution

A. Maier, A. Sindel, S. Zeitler

Pattern Recognition Lab, Friedrich-Alexander-Universität Erlangen-Nürnberg

December 09, 2024

1. Software evolution
2. Evolution processes
3. Change Management
4. Software maintenance
4.1 Maintenance prediction
4.2 Software reengineering
5. Legacy systems

What is software evolution?

Lifespan of software systems[4]

Usually large software systems have a long lifetime due to their high cost.

While business systems are often more than 10 years old, military systems can have a lifespan of 30 years and more.

Here are some exemplary softwares and their initial release dates:

Software	Release date
SABRE Airline Reservation System	1960 [1]
IRS Individual Master File and Business Master File	1962 [2]
Microsoft Word	1983 [3]

[1] Guinness World Records: *Oldest software system in continuous use* (2023) [1]

[2] Guinness World Records: *Oldest software system in continuous use* (2023) [1]

[3] Wikimedia: *History of Microsoft Word* (2023) [2]

[4] Sommerville: *Software Engineering* (2016) [3] p. 256

What is software evolution?

Changes in a software environment[5]

To receive a return of investment, large software needs to stay useful despite

- business changes
- changing of user expectations
- changing of hardware or software platforms
- changes in the state of the art

Software evolution is the process of adapting software to these changes.

[5] Sommerville: *Software Engineering* (2016) [3] p. 256

1. Software evolution
2. Evolution processes
3. Change Management
4. Software maintenance
4.1 Maintenance prediction
4.2 Software reengineering
5. Legacy systems

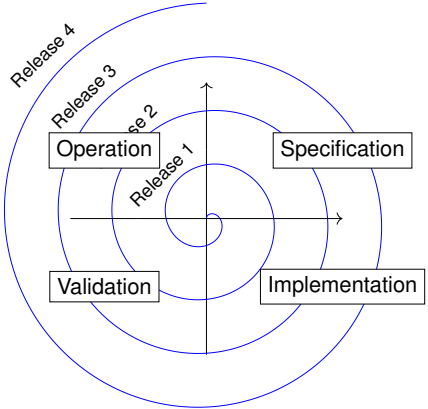
Software evolution

Software evolution

Spiral process[6]

Software evolution can be described as a spiral process

- Requirements change with business or environment change
- New releases incorporate changes in regular intervals
- Need for changes may become obvious before deployment
  $\rightarrow$ Development of next release may start before release of current version
- Due to competitive pressure time between releases may be weeks rather than years



Source: Sommerville 2016 [3] p. 257 (adapted)

[6] Sommerville: *Software Engineering* (2016) [3] p. 256-257

Software evolution

Custom software[7]

For custom software, the customer pays for development and then takes over evolution or issues a separate contract.

This may lead to discontinuities due to

- Documents not being passed on
- Inheritance of software due to reorganization of companies

A non-smooth transition is called **software maintenance** and includes activities such as program understanding.

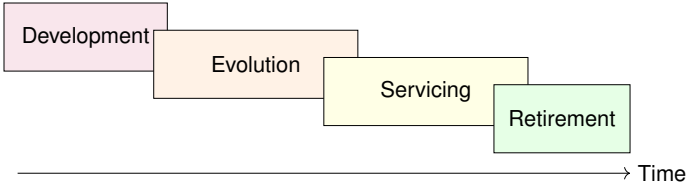
[7] Sommerville: *Software Engineering* (2016) [3] p. 257

Software evolution

Servicing[8]

An alternative model of software evolution is based on evolution and servicing

- Evolution: Changes to the requirements are proposed and implemented
  → Modifications degrade the system structure
  → Changes become increasingly expensive
- Servicing: Only small changes are made due to cost of changes
  → Replacing and retiring the system starts to be discussed



Source: Sommerville 2016 [3] p. 258 (adapted)

[8] Sommerville: *Software Engineering* (2016) [3] p. 257-258

1. Software evolution
2. Evolution processes
3. Change Management
4. Software maintenance
4.1 Maintenance prediction
4.2 Software reengineering
5. Legacy systems

Evolution processes

Evolution process[9]

The evolution process depends on:

- type of software
- skills of people
- used process during development
- organizational standards

Change proposals, either formal or informal, are the driver for system evolution.

[9] Sommerville: *Software Engineering* (2016) [3] p. 258-259

Evolution processes

Change proposals[10]

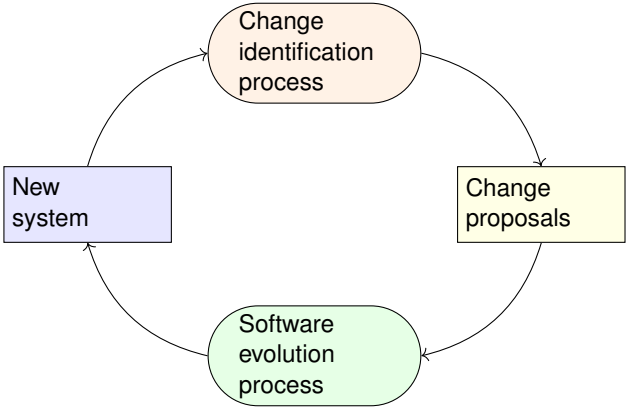
Change proposals can be based on:

- existing requirements
- requests for new requirements
- bug reports
- software improvement

[10] Sommerville: *Software Engineering* (2016) [3] p. 258-259

Evolution processes

Evolution processes



Source: Sommerville 2016 [3] p. 259 (adapted)

Evolution processes

Change identification process[11]

The software needs to be analyzed before a change proposal is accepted.

This analysis is used to:

- figure out which components need to be changed
- estimate the cost of change
- estimate the impact of change
- ensure the correct versions of components are included in the release

[11] Sommerville: *Software Engineering* (2016) [3] p. 258-259

Evolution processes

Software evolution process[12]

The software evolution process includes a wide range of fundamental activities

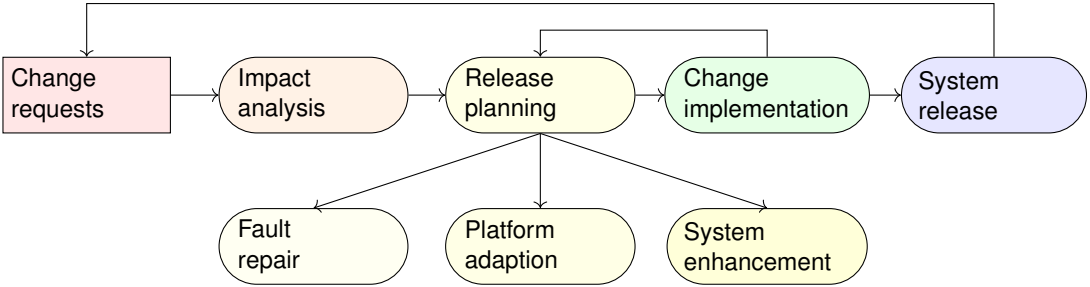
- When change is requested, it's impact and cost are analyzed
- If the changes are accepted, a new release is planned
- During release planning, all changes are considered and some are selected for the next release

[12] Sommerville: *Software Engineering* (2016) [3] p. 259

Evolution processes

Software evolution process[13]

A generic model of the software evolution process:



Source: Sommerville 2016 [3] p. 259 (adapted)

[13] Sommerville: *Software Engineering* (2016) [3] p. 259

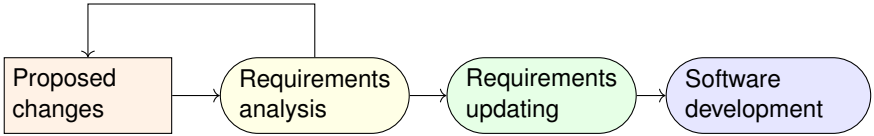
Evolution processes

Change implementation[14]

The change implementation process depends on how tightly development and evolution are integrated.

- Team remains the same → evolution is another iteration of development
- Different teams → a program understanding phase is required to understand
  - the program structure
  - how functionality is delivered
  - the affect of changes to the program

  and ensure that changes do not cause problems in the existing system.



Source: Sommerville 2016 [3] p. 260 (adapted)

[14] Sommerville: *Software Engineering* (2016) [3] p. 259-260

Evolution processes

Change implementation[15]

Sometimes urgent changes arise due to

- serious system fault
- changes in the operating environment
- unanticipated changes in the business (such as new competitors)

In these cases, an emergency repair is needed.

This often means that documentation is not updated and requirements, design, and code become inconsistent.

Agile approaches aim to resolve this problem by maintaining minimal documentation.

[15] Sommerville: *Software Engineering* (2016) [3] p. 260

Evolution processes

Emergency repair process[16]

Emergency repairs

- are done as quickly as possible
- provide a quick, workable solution rather than the best solution
- accelerate software aging

After an emergency repair the code should be refactored to avoid degrading of the software.

Source: Sommerville 2016 [3] p. 260 (adapted)

[16] Sommerville: *Software Engineering* (2016) [3] p. 260-261

Evolution processes

Agile and plan-driven evolution[17]

Problems may arise if the development and evolution teams have different members as well as different approaches:

- Agile development and plan-driven evolution:
  - There is no detailed documentation for the evolution team
  - Clearly defined requirements might not exist for the evolution team
- Plan-driven development and agile evolution:
  - Evolution team needs to develop automated tests from scratch
  - Code might not be refactored and simplified $\rightarrow$ reengineering is required

[17] Sommerville: *Software Engineering* (2016) [3] p. 261

Evolution processes

Agility and Evolution processes[18]

Agile techniques and methods have some advantages and disadvantages for system evolution:

Pros

- Test-driven development
- Automated regression testing
- System changes as user stories

Cons

- Involving users in the development team is practically impossible
- Emergency repairs might interrupt development cycles
- Gap between releases may be lengthened to not disrupt operational process

[18] Sommerville: *Software Engineering* (2016) [3] p. 261

1. Software evolution
2. Evolution processes
3. Change Management
4. Software maintenance
4.1 Maintenance prediction
4.2 Software reengineering
5. Legacy systems

Change Management

Change management

Prototyping[19]

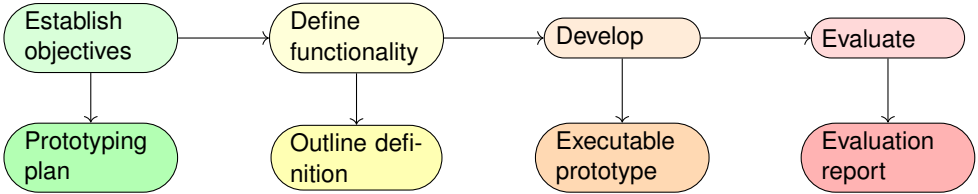
A prototype is an early version of a software used to

- demonstrate concepts
- try design options
- help understand the problem
- find possible solutions
- **explore solutions** and user interfaces
- **select and validate requirements**
- control costs
- refine requirements

[19] Sommerville: *Software Engineering* (2016) [3] p. 62

Change management

Prototyping



Source: Sommerville 2016 [3] p. 63 (adapted)

Change management

Incremental delivery[20]

With the use of incremental delivery, the system is split into smaller portions which are delivered by priority. Unlike prototypes, these increments are part of the real system.

Pros

- Customer defines the priority of services
- Software can be used immediately
- Customer gains experience with the system
- Customer can refine requirements for later increments
- Changes are easy to incorporate
- Most important system parts are tested the most

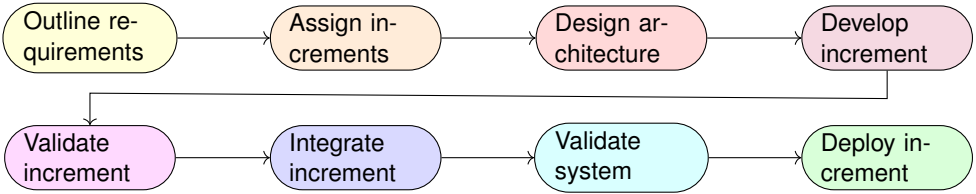
Cons

- Software basis can not be identified without detailed requirements
- Can conflict with organizational structures, especially contracts
- Unsuitable to replace existing systems

[20] Sommerville: *Software Engineering* (2016) [3] p. 64-65

Change management

Incremental delivery



Source: Sommerville 2016 [3] p. 64 (adapted)

1. Software evolution

2. Evolution processes

3. Change Management

4. Software maintenance

4.1 Maintenance prediction

4.2 Software reengineering

5. Legacy systems

Software maintenance

What is Software maintenance?[21]

"The purpose of the Maintenance process is to sustain the capability of the system to provide a service"[4]

"For software systems, the Maintenance process makes corrections, changes, and improvements to deployed software systems and elements."[4]

[21] Sommerville: *Software Engineering* (2016) [3] p. 270-271

Software maintenance

What is Software maintenance?[22]

The term **Software maintenance** is usually applied when separate teams are involved during and after development.
There are different types of software maintenance:

- **Fault repair**: Fix bugs and vulnerabilities
  Note: Also called “Corrective maintenance"
- **Environmental adaptation**: Adapt software to new environments
- **Functionality addition**: Add new features and support new requirements

In practice, maintenance is a mix of these types.

[22] Sommerville: *Software Engineering* (2016) [3] p. 270-271

Software maintenance

Cost of maintenance[23]

It is generally more expensive to add features during maintenance because:

- The new team needs to understand the system
- Development teams have to incentive to write maintainable code
- Program maintenance is unpopular work
- Structures degrade with system age

[23] Sommerville: *Software Engineering* (2016) [3] p. 272-273

Software maintenance

Cost of maintenance[24]

There are two main reasons why businesses are reluctant to increase development cost to reduce long term maintenance cost:

- Managers are usually incentivized to reduce short-term costs
  - Investment into maintenance leads to a measurable increase in short term cost
  - The long term gains are not easily measurable
- Developers are often not responsible for maintenance
  - Developing for better maintainability is additional work
  - They do not benefit from this increased workload

[24] Sommerville: *Software Engineering* (2016) [3] p. 274

Maintenance prediction

Software maintenance

Maintenance prediction[25]

Maintenance prediction tries to estimate

- the changes that will be required
- the cost of changes

By identifying the component that will be the most costly to change, investments can be made to reduce the maintenance cost of these components.

For maintenance prediction, you need to predict:

- Maintainability
- System changes
- Maintenance cost

[25] Sommerville: *Software Engineering* (2016) [3] p. 274-275

Software maintenance

Maintenance prediction[26]

To predict the number of change requests, the system's relationship with the environment needs to be understood by evaluating the systems

- Number and complexity of interfaces
- Number of volatile requirements
- Used business processes

Maintenance cost is related to the complexity of the system. By identifying complex parts and reducing the complexity, maintenance costs can be reduced.

[26] Sommerville: *Software Engineering* (2016) [3] p. 274-275

Software maintenance

Maintenance prediction[27]

Once the system is deployed, process data can be used to predict maintenance.
Process metrics used to assess maintainability include

- Number of requests for corrective maintenance
- Average time taken for impact analysis
- Average time taken to implement change request
- Number of outstanding change requests

[27] Sommerville: *Software Engineering* (2016) [3] p. 275-276

Software reengineering

Software maintenance

Software reengineering[28]

Especially legacy systems are often difficult to understand and thus hard to maintain.

Reengineering a system improves structure and understandability, making the system easier to maintain.

Reengineering may involve

- Redocumenting
- Small changes to the architecture
- Translating code to modern language
- Modifying or updating structures and values of the data

During these activities, the functionality of the system is not changed. Major changes to the architecture should be avoided.

[28] Sommerville: *Software Engineering* (2016) [3] p. 276

Software maintenance

Software reengineering[29]

Reengineering has two major advantages compared to replacement

- Reduced risk
  - Redevelopment is a high risk
  - Errors may be made during specification
  - Problems can arise during development
  - Delays lead to lost business and extra cost
- Reduced cost

During these activities, the functionality of the system is not changed. Major changes to the architecture should be avoided.

[29] Sommerville: *Software Engineering* (2016) [3] p. 276-277

Software maintenance

Software reengineering[30]

Software reengineering includes the following activities:

Source code translation

- Conversion to a modern programming language
- Automated using a translation tool

Reverse engineering

- Analysis of the program to document organization and functionality
- Usually automated

Program structure improvement

- Analysis of the control structure and modifications to improve readability
- Can be automated, but manual intervention is required

[30] Sommerville: *Software Engineering* (2016) [3] p. 277

Software maintenance

Software reengineering[31]

Program modularization

- Grouping of related parts to remove redundancy, may include architectural refactoring
- Manual process

Data reengineering

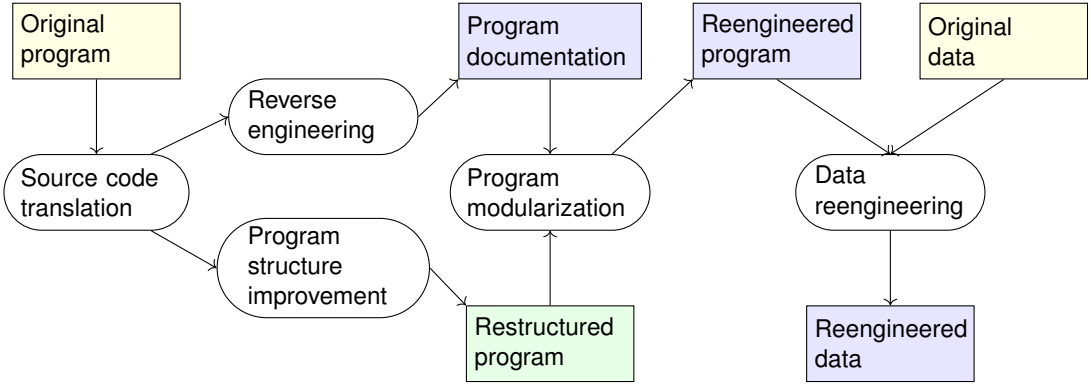
- Changing program data to reflect program changes by redefining and converting databases
- Expensive, time intense process to find and correct mistakes

Some of these activities may not be required. If the application language is still used, translation is not necessary. Data reengineering is not required if the data structures remain the same.

[31] Sommerville: *Software Engineering* (2016) [3] p. 277

Software maintenance

Software reengineering



Source: Sommerville 2016 [3] p. 277 (adapted)

Software maintenance

Software reengineering[32]

While reengineering is usually cost efficient, it has limitations:

- Functionally written code can not be converted to object oriented
- Major architecture changes can not be automated and are thus expensive
- Reengineered systems are not as maintainable as new systems

[32] Sommerville: *Software Engineering* (2016) [3] p. 278

Software maintenance

Refactoring[33]

Refactoring aims to slow program degradation through changes.
Modifications are made to

- improve structure
- reduce complexity
- improve understandability

During refactoring, there are no new functionalities added.

Refactoring is part of agile methods, but does not depend on them.

[33] Sommerville: *Software Engineering* (2016) [3] p. 278

Software maintenance

Refactoring[34]

While reengineering is only done once a system has been maintained, refactoring is a contiuous process to avoid degradation of the system. Refactoring includes the improvement of

- Code duplicates: Implement a single method or function instead
- Long methods: Split into a number of shorter methods
- Switch case statements: Use polymorphism instead
- Data clumping: The same group of items in several places are replaced by one object that encapsulates all data
- Speculative generality: Remove generality that is not used

Refactoring effectively reduces the long-term maintenance cost of a system.
Significantly degraded programs may also require the refactoring of the design, which includes identifying relevant patterns and replacing code with these patterns.

[34] Sommerville: *Software Engineering* (2016) [3] p. 279

1. Software evolution
2. Evolution processes
3. Change Management
4. Software maintenance
4.1 Maintenance prediction
4.2 Software reengineering
5. Legacy systems

Legacy systems

What are legacy systems?[35]

Legacy systems are older systems that rely on outdated languages and technology.

Legacy software often:

- has been maintained over a long time period
- has a degraded structure
- is dependent on old hardware
- is associated with legacy processes
- is impossible to modify for new processes

[35] Sommerville: *Software Engineering* (2016) [3] p. 261-262

Legacy systems

As part of a bigger system[36]

Legacy systems are broader socio-technical systems including:

System hardware: Hardware a legacy system may be based on

- Maintenance is expensive
- May be incompatible with current IT purchasing policies

Support software: Software that a legacy system may rely on

- e.g. operating system, utilities, compilers
- May be no longer supported by original providers

[36] Sommerville: *Software Engineering* (2016) [3] p. 262

Legacy systems

As part of a bigger system[37]

Application software: Provides business services

- Made up of a number of application programs
- Often developed at different times
- Shared across application software systems

Application data: Data processed by the application system

- For legacy systems there are often immense volumes of data accumulated
- This data may be inconsistent, duplicated, or spread across databases

[37] Sommerville: *Software Engineering* (2016) [3] p. 262

Legacy systems

As part of a bigger system[38]

Business processes: Processes used in the business to achieve a business objective

- May be designed around a legacy system
- May be constrained by functionality a legacy system provides

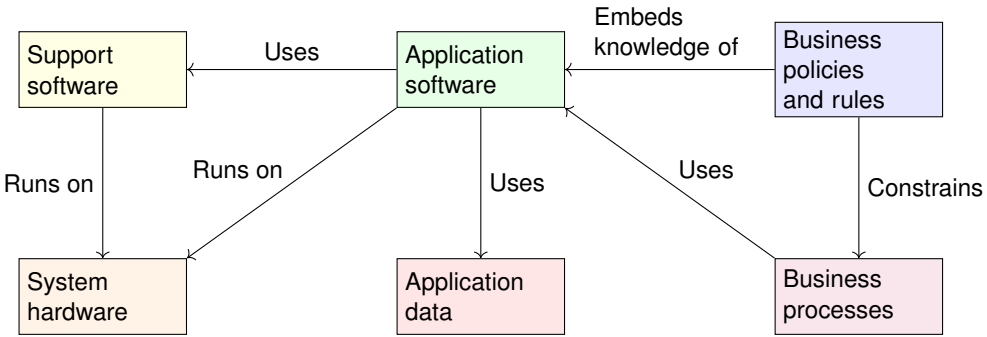
Business policies and rules: Definitions how the business should be carried out

- Constrains the business
- Use of legacy application systems may be embedded

[38] Sommerville: *Software Engineering* (2016) [3] p. 262

Legacy systems

As part of a bigger system[39]



Source: Sommerville 2016 [3] p. 260 (adapted)

[39] Sommerville: *Software Engineering* (2016) [3] p. 263

Legacy systems

As a layered system[40]

A legacy system can also be illustrated using layers:

Business processes
Application software
Platform and infrastructure software
Hardware

Source: Sommerville 2016 [3] p. 264 (adapted)

- Each layer depends on the one immediately below
- If interfaces are maintained, changes don't affect other layers
- In practice changes to one layer may affect layers above and below due to
  - Introduction of new facilities
    $\rightarrow$ Higher layers may need changes
  - Changes that slow down software
    $\rightarrow$ New hardware is required
    $\rightarrow$ New changes are possible
  - Introduction of new hardware
    $\rightarrow$ Hardware interfaces change

[40] Sommerville: *Software Engineering* (2016) [3] p. 262-264

Legacy systems

Changing legacy systems[41]

Changes to legacy systems are expensive because

- program styles and conventions are inconsistent, making it difficult to understand
- obsolete programming languages may be used, requiring a specialist
- system documentation is often out of date or not available
- system structure has degraded due to maintenance
- optimization for older hardware makes it hard to understand
- data may be inconsistent, duplicated, inaccurate, or incomplete

[41] Sommerville: *Software Engineering* (2016) [3] p. 265

Legacy systems

Changing legacy systems[42]

Despite the cost and complexity of legacy systems, they are still widely used because the risk of replacing them is too big.

Replacing legacy systems is both expensive and risky because:

- Complete specifications rarely exist
- Legacy systems and business processes may be intertwined
- Important business rules may be embedded in the software and undocumented
- New software development is risky, costs and time may exceed the initial estimate

[42] Sommerville: *Software Engineering* (2016) [3] p. 264

Legacy systems

Legacy system management[43]

There are four strategies to manage legacy systems:

Remove it completely: When system has no effective contribution to the business process

Leave it unchanged and continue maintenance

- System is still required
- System is relatively stable with few change requests

Reengineering to improve maintainability

- System quality has degraded, but new changes are required
- Development of new interface components

[43] Sommerville: *Software Engineering* (2016) [3] p. 266

Legacy systems

Legacy system management[44]

Replace whole or parts with a new system

- Old system can not continue operation
- E.g. due to hardware, off-the-shelf solutions, etc.
- Evolutionary replacement can often be adopted

To decide on the correct strategy, the legacy system needs to be assessed from a business and technical perspective.

[44] Sommerville: *Software Engineering* (2016) [3] p. 267

Legacy systems

Business assessment[45]

To assess the business value of a system, the identification of system stakeholders is needed.

The stakeholders then need to state

- How often and by how many people is the system used
- The business processes the system supports and their value
- Is the system dependable or does it cause problems
- Importance of the systems outputs

[45] Sommerville: *Software Engineering* (2016) [3] p. 266

Legacy systems

Technical assessment[46]

To assess the technical quality a system, both application software and environment need to be considered including hardware, support software, compiler, debugger and development environments needed for maintenance.
The environment can be assessed with

- Supplier stability: Is the supplier still in business? How likely will he retire? Are there alternatives?
- Failure rate: Rate of reported failures, crashes and force restarts
- Age: The older the software, the more obsolete
- Performance: Is the performance adequate or does us effect the users?
- Support requirements: Local support required by hard- and software and it's cost
- Maintenance cost: Cost of hardware maintenance and support software licenses
- Interoperability: Interface problems, version problems

[46] Sommerville: *Software Engineering* (2016) [3] p. 268-269

Legacy systems

Technical assessment[47]

For the assessment of the application system, you should consider:

- Understandability: Complexity of structures, Variable names, Readability
- Documentation: Completeness, consistency, currency
- Data: Consistency, completeness, duplication across files
- Performance: Effect of performance problems on the user
- Programming language: Availability of modern compilers, Usage for new systems

[47] Sommerville: *Software Engineering* (2016) [3] p. 269-270

Legacy systems

Technical assessment[48]

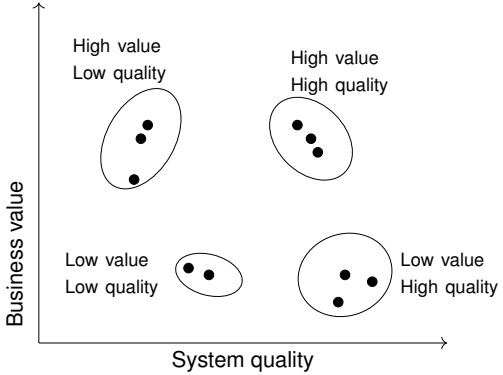
- Configuration management: How are configurations managed? Are current versions documented?
- Test data: Availability, Documentation
- Personnel skills: Availability of skilled personal for maintenance, Experience
- Number of change requests: The more change requests, the lower the quality
- Number of user interfaces: The more interfaces, the likelier are inconsistency and redundancy
- Volume of data used: The more data, the more inconsistencies and errors

[48] Sommerville: *Software Engineering* (2016) [3] p. 269-270

Legacy systems

Legacy system assessment[49]

Exemplary legacy system assessment:



Source: Sommerville 2016 [3] p. 267 (adapted)

- Low quality, low value:
  System should be removed
- Low quality, high value:
  System should be reengineered or replaced with off-the-shelf systems
- High quality, low value:
  If it's not worth replacing, maintenance may continue. If maintenance becomes too expensive, it should be removed.
- High quality, high value:
  System should be kept in operation, normal maintenance should continue

[49] Sommerville: *Software Engineering* (2016) [3] p.

Questions & References

Comprehensive questions

- What is the difference between software evolution and servicing?
- Which processes are part of the software evolution process?
- How and when is change implemented?
- What are legacy systems and with which other systems do they interact?
- Which factors are important to estimate the cost of maintenance?
- What is the difference between refactoring and reengineering?

Further reading

- Ian Sommerville, Software Engineering, [3], 2016
- Anja Metzner, Software Engineering - kompakt, [5], 2020

References I

[1] Limited Guinness World Records. *Oldest software system in continuous use*. 2023. URL: https://www.guinnessworldrecords.com/world-records/636196-oldest-software-system-in-continuous-use (visited on 02/01/2024).

[2] Inc. Wikimedia. *History of Microsoft Word*. 2023. URL: https://en.wikipedia.org/wiki/History_of_Microsoft_Word (visited on 02/01/2024).

[3] Ian Sommerville. *Software Engineering*. Always learning. Pearson, 2016.

[4] "ISO/IEC/IEEE International Standard - Software engineering - Software life cycle processes - Maintenance". In: *ISO/IEC/IEEE 14764:2022(E)* (2022), pp. 1–46.

[5] Anja Metzner. *Software Engineering - kompakt*. Carl Hanser Verlag GmbH & Company KG, 2020.